

#1 1 Sliding Window, Two Pointer, Prefix Sum

김성수

4thrunyul@gmail.com

0. 목차



1. Sliding Window

2. Two Pointer

3. Prefix Sum

1. Sliding Window

1. Sliding Window



Sliding Window

- 1차원 배열에서 고정된 크기의 구간(창문)을 한 칸씩 밀어 이동시키면서, 범위에서 벗어나는 원소와 새로 들어오는 원소만으로 내부 데이터를 효율적으로 갱신하는 알고리즘
- 고정성 : 탐색 대상이 되는 구간의 크기 K 가 항상 일정하게 유지됨
- 연속성 : 배열이나 문자열처럼 연속된 데이터 구조에서 특정 범위의 상태를 다룰 때 사용
- 효율성
 - 매 이동마다 다음 원소 추가와 이전 원소 제거, 단 2번의 연산만 수행하여
 - $O(n * k)$ 를 $O(n)$ 으로 단축
- 재사용성 : 이전 구간의 결과(합, 개수 등)를 버리지 않고 재활용하여 중복 연산 제거
- 문제 유형
 - 길이가 N 인 수열에서 연속된 K 의 구간의 합의 최대 혹은 최소 찾기

1. Sliding Window

문제 상황

- 길이가 N인 수열에서 연속된 K의 구간의 합의 최대값 찾기
- $n = 7, k = 3$
- 기존의 방식 $O(n * k)$

배열 v[7]	5	3	-7	2	1	9	4
---------	---	---	----	---	---	---	---

$$[0:2] \quad 5 + 3 - 7 = 1$$

$$[3:5] \quad 2 + 1 + 9 = 12$$

$$[1:3] \quad 3 - 7 + 2 = -2$$

$$[4:6] \quad 1 + 9 + 4 = 14$$

$$[2:4] \quad -7 + 2 + 1 = -4$$

1. Sliding Window

문제 상황

- Sliding Window 사용 : $O(n)$
- $n = 7, k = 3$
- $\text{sum} = [0:2] = 1$

배열 $v[7]$

5	3	-7	2	1	9	4
---	---	----	---	---	---	---

$$\begin{aligned}\text{sum} &+= v[3] - v[0] \\ 1 &+= 2 - 5\end{aligned}$$

배열 $v[7]$

5	3	-7	2	1	9	4
---	---	----	---	---	---	---

$$\text{sum} = -2$$

1. Sliding Window

문제 상황

- Sliding Window 사용 : $O(n)$
- $n = 7, k = 3$
- $\text{sum} = [0:2] = 1$

배열 $v[7]$

5	3	-7	2	1	9	4
---	---	----	---	---	---	---

$$\begin{aligned}\text{sum} &+= v[4] - v[1] \\ -2 &+= 1 - 3\end{aligned}$$

배열 $v[7]$

5	3	-7	2	1	9	4
---	---	----	---	---	---	---

$$\text{sum} = -4$$

1. Sliding Window: 실습



실습

- [2524 가장 큰 합](#)
- [2497 수열](#)

2. Two Pointer

2. Two Pointer



Two Pointer

- 1차원 배열에서 두 개의 포인터(인덱스 위치)를 조건에 따라 독립적으로 이동시키면서, 원하는 조건을 만족하는 구간이나 값의 조합을 효율적으로 탐색하는 알고리즘
- 가변성 : 고정 크기를 갖는 슬라이딩 윈도우와 달리, 두 포인터 사이의 구간 크기가 조건 충족 여부에 따라 확장되거나 축소됨
- 단조성 : 두 포인터가 한쪽 방향으로만 전진하거나(되돌아가지 않음) 정렬 상태를 활용하므로, 이미 지나간 불필요한 탐색 범위를 건너뛴
- 효율성 : 완전 탐색 시 $O(n^2)$ 이 소요되는 2중 루프 탐색을 두 포인터 이동 횟수의 총합인 최대 $2n$ 번 연산 $O(n)$ 으로 최적화
- 문제 유형
 - 정렬된 배열에서 두 수의 합 / 차 찾기
 - 조건을 만족하는 가변 길이의 연속 부분 수열 탐색

2. Two Pointer

유형 1 : 같은 방향으로 전진

- 문제 : 길이가 n 인 배열에서 연속 수열의 합이 t 가 되는 구간이 몇 개인지 구하기
 - 초기 조건 : $left = 0, right = 0, sum = v[0], count = 0$
 - 필수 조건 : 배열 안에 포함된 원소가 모두 0 이상이어야 함(음수 불가능)
 - 핵심 원리
 - $sum < t$
 - sum 의 값이 부족하므로 $++right$ 이후 $sum += v[right]$ 수행
 - $sum > t$
 - sum 의 값이 넘치므로 $sum -= v[left]$ 이후 $++left$ 수행
 - $sum == t$
 - 조건을 만족하므로 $++count$
 - 다른 수열을 찾기 위해 $s -= v[left]$ 이후 $++left$
 - 종료 조건 : $right == n$ 이면서 $sum < t$ 인 경우

2. Two Pointer

유형 2 : 양 끝에서 좁혀오기

- 문제 : 정렬된 길이가 n 인 배열에서 두 원소의 합이 t 가 되는 쌍의 개수를 구하기
 - 초기 조건 : $left = 0, right = n - 1, sum = v[0] + v[n - 1], count = 0$
 - 필수 조건 : 배열이 오름차순으로 정렬되어 있어야 함, 모든 원소는 다른 수
 - 핵심 원리
 - $sum < t$
 - 두 수의 합이 부족하므로 $sum -= v[left], ++left, sum += v[left]$ 수행
 - $sum > t$
 - 두 수의 합이 넘치므로 $sum -= v[right], --right, sum += v[right]$ 수행
 - $sum == t$
 - 조건을 만족하므로 $++count$ 수행 후 $left$ 와 $right$ 한 칸 씩 이동
 - $sum -= v[left], ++left, sum += v[left]$
 - $sum -= v[right], --right, sum += v[right]$
 - 종료 조건 : $left \geq right$ 가 되는 순간 탐색 종료

2. Two Pointer: 실습



실습

- [2300 용액](#)
- [13656 연속된 소수의 합](#)

3. Prefix Sum

3. Prefix Sum



문제 상황

- 크기가 n 인 정수 배열 v 에서 인덱스 L 부터 R 까지의 합을 구하라
- 단순 for문을 통해 해결 가능, $O(n)$

배열 $v[4]$	5	3	-7	2
-----------	---	---	----	---

$$\text{TC1 : } L = 0, R = 2 \qquad 5 + 3 - 7 = 1$$

$$\text{TC2 : } L = 2, R = 4 \qquad 3 - 7 + 2 = -2$$

$$\text{TC3 : } L = 3, R = 4 \qquad -7 + 2 = -5$$

3. Prefix Sum



문제 상황

- 크기가 n 인 정수 배열 v 가 있다.
- 인덱스 L 부터 R 까지의 합을 구하라는 명령 Q 가 m 번 주어진다.
- $O(n * m)$

배열 $v[4]$	5	3	-7	2
-----------	---	---	----	---

	$L = 0, R = 2$	$5 + 3 - 7 = 1$
TC1	$L = 1, R = 3$	$3 - 7 + 2 = -2$
	$L = 3, R = 4$	$-7 + 2 = -5$

3. Prefix Sum

Prefix Sum

- 배열의 시작부터 특정 위치까지의 합을 미리 저장한 배열

$$s[i] = v[i] + s[i - 1]$$

- $s[i - 1]$ 을 인덱싱 할 때 Out of Bounds 오류가 생길 수 있음
- `vector<int> v(n + 1)`와 `vector<int> s(n + 1)`

배열 $v[5]$

0	5	3	-7	2
---	---	---	----	---

배열 $s[5]$

0	5	8	1	3
---	---	---	---	---

3. Prefix Sum

Prefix Sum

- 인덱스 L과 R 만큼의 합 구하기

$$\sum(l, r) = s[r] - s[l - 1]$$

- 누적합 배열 생성 : $O(n)$
- 인덱스 L부터 R까지의 합을 구하라는 명령 Q가 m번 : $O(m)$
- $O(n + m)$

배열 v[5]

0	5	3	-7	2
---	---	---	----	---

배열 s[5]

0	5	8	1	3
---	---	---	---	---

예제

- $L = 2, R = 4$
- $s[4] = 5 + 3 - 7 + 2$
- $s[1] = 5$
- $s[R] - s[L - 1]$
- $s[4] - s[1] = 5 + 3 - 7 + 2 - 5 = -2$

3. Prefix Sum



- [3135 Tutorial : const 구간의 합 구하기 \(1D\)](#)
 - 코드

3. Prefix Sum

2차원 배열에서의 Prefix Sum

$$s[i][j] = v[i][j] + s[i-1][j] + s[i][j-1] - s[i-1][j-1]$$

배열 $v[5][5]$

0	0	0	0	0
0	1	2	3	4
0	5	6	7	8
0	9	10	11	12
0	13	14	15	16

배열 $s[5][5]$

0	0	0	0	0
0	1	3	6	10
0	6	14	24	36
0	15	33	54	78
0	28	60	96	136

3. Prefix Sum

2차원 배열에서의 Prefix Sum

$$s[i][j] = v[i][j] + s[i-1][j] + s[i][j-1] - s[i-1][j-1]$$

배열 $v[5][5]$

0	0	0	0	0
0	1	2	3	4
0	5	6	7	8
0	9	10	11	12
0	13	14	15	16

배열 $s[5][5]$

0	0	0	0	0
0	1	3	6	10
0	6	14	24	36
0	15	33	54	78
0	28	60	96	136

예제

- $s[3][3] = v[3][3] + s[2][3] + s[3][2] - s[2][2]$
 - $s[2][3] = 1 + 2 + 3 + 5 + 6 + 7$
 - $s[3][2] = 1 + 2 + 5 + 6 + 9 + 10$
- => 중복으로 포함된 1, 2, 5, 6을 뺄 필요가 있음

3. Prefix Sum

2차원 배열에서의 Prefix Sum

$$\sum((r_1, c_1), (r_2, c_2)) = s[r_2][c_2] - s[r_1 - 1][c_2] - s[r_2][c_1 - 1] + s[r_1 - 1][c_1 - 1]$$

배열 v[5][5]

0	0	0	0	0
0	1	2	3	4
0	5	6	7	8
0	9	10	11	12
0	13	14	15	16

배열 s[5][5]

0	0	0	0	0
0	1	3	6	10
0	6	14	24	36
0	15	33	54	78
0	28	60	96	136

예제

- sum((2,2), (3, 3))
- $s[3][3] - s[1][3] - s[3][1] + s[1][1]$

3. Prefix Sum



- [3136 Tutorial : const 구간의 합 구하기 \(2D\)](#)
 - 코드

3. Prefix Sum: 실습



실습

- [3706 합이 0이 되는 연속구간 세기](#)
- [5656 보석 괴물](#)